

מסמך סקייל בסיסי – HavuGym Crew

מכסה את סעיף 8 בבריף. כל המספרים כאן נמדדו על הפריסה החיה, לא הוערכו.

1. צורת העומס

לפני מספרים – הבנת הצורה, כי היא קובעת מה בכלל מסוכן.

המוצר מחולק מטבעו לחבורות של 3-10 חברים. אין שאילתה אחת שנוגעת בכל המשתמשים. כל קריאה מסוננת ל-tenant יחיד, בין אם הקוד ביקש זאת ובין אם RLS אכף זאת. משמעות מעשית: **המערכת מתרחבת אופקית לפי חבורות, לא לפי משתמשים.** אלף חבורות אינן אלף פעמים יותר קשות משאילתה על חבורה אחת – הן אלף שאילתות עצמאיות, כל אחת בגודל של חבורה.

דפוס	תדירות	עלות
קריאת feed	הכי גבוהה	אגרציה על אימוני חבורה + הגעות פעילות
באנר הגעות	כל דקה, לכל לשונית פתוחה	סריקת אינדקס חלקי; שורות בודדות
רישום אימון	~3 בשבוע למשתמש	כתיבה בטרנזקציה + חישוב ציון
חיפוש תרגיל	כמה בכל רישום	חיפוש טקסט על 660 שורות
דף חבורה	יומית	אגרציה שבועית + סגירה עצלה
סגירת שבוע	פעם בשבוע לחבורה	0(חברים)

2. מדידות בפועל

EXPLAIN (ANALYZE, BUFFERS) על הפריסה החיה. הנתונים הנוכחיים: 44 אימונים, 363 סטים, 660 תרגילים, 9 משתמשים.

שאילתה	זמן ביצוע	מה התוכנית עושה
feed של חבורה, עמוד ראשון	10.5ms	GroupAggregate על 35 שורות → Limit → Sort
סטטיסטיקה שבועית לחבורה	3.6ms	GroupAggregate
חיפוש תרגיל ('ilike '%bench%')	3.4ms	Seq Scan על 660 שורות
בייסליין אישי (8 אחרונים)	0.57ms	
/api/health / מקצה לקצה מ-fra1	375ms-167	כולל שתי ספירות ובדיקת RLS

שימוש באינדקסים בפועל (pg_stat_user_indexes) מאשר שהאינדקסים שתוכננו הם אלה שנמצאים בשימוש:

exercises_pkey	2873	
havura_members_pkey	1165	החם RLS-נתיב ה ←
workouts_pkey	502	
workout_sets_workout_idx	419	

3. הבעיה האמיתית שהמדידה חשפה

זה הממצא המרכזי של המסמך, והוא לא היה ידוע לפני שהרצתי EXPLAIN.

ה-feed נקרא מ-view שמבצע GROUP BY כדי לחשב מספר סטים, נפח ורשימת שרירים לכל אימון. התוכנית מראה:

```
Sort (rows=21)
└─ GroupAggregate (rows=35) ← כל אימוני החבורה = 35
```

ה-LIMIT 21 לא נדחף מתחת ל-Postgres GROUP BY. מאגר את כל אימוני החבורה ורק אחר כך ממין וחותר. עם 35 אימונים זה 10ms. עם 10,000 אימונים בחבורה ותיקה, כל טעינת feed תעבוד על 10,000 שורות כדי להציג 20.

זה עומס ליניארי בהיסטוריה של החבורה על הפעולה הנפוצה ביותר במוצר – בדיוק הדפוס שנראה תקין בפיתוח ונשבר בשנה השנייה.

3.1 למה לא תיקנתי את זה עכשיו

התיקון ידוע ואינו קשה: לחשב set_count ו-volume בזמן הכתיבה ולשמור אותם על workouts. הכתיבה כבר מחשבת ציון באותה טרנזקציה, אז זו אותה עבודה במקום שכבר קיים, וה-feed הופך לסריקת אינדקס טהורה על (havura_id, performed_at desc) עם LIMIT שנדחף עד הסוף.

לא ביצעתי כי זו דנורמליזציה שדורשת backfill ומיגרציה נוספת, והנתונים הנוכחיים לא מצדיקים אותה. זו החלטה מודעת עם תאריך תפוגה ידוע: היא הופכת נחוצה סביב אלפי אימונים לחבורה. עדיף למדוד ולהצהיר מאשר לאופטם מראש דבר שאולי לא היה נשבר.

4. Pagination

ה-feed משתמש ב-keyset, לא ב-offset:

```
let query = supabase.from('workout_feed')
  .eq('havura_id', havuraId)
  .order('performed_at', { ascending: false })
  .limit(limit + 1);
if (before) query = query.lt('performed_at', before);
```

- OFFSET 1000 מכריח את Postgres לייצר ולזרוק 1000 שורות. עמוד 50 עולה פי חמישים מעמוד 1.
- Keyset מקבל את המיקום כערך (performed_at < cursor) ומשתמש באינדקס ישירות. עמוד 50 עולה כמו עמוד 1.
- limit + 1 – שורה אחת נוספת נמשכת רק כדי לדעת אם יש עוד עמוד, בלי COUNT(*). נפרד.

4.1 ה-cursor הוא זוג, לא חותמת זמן

בגרסה הראשונה ה-cursor היה `performed_at` בלבד, וזו הייתה תקלה אמיתית: שני חברים שסיימו אימון באותה דקה מייצרים אותה חותמת בדיוק, ותנאי `performed_at < last` היה **מדלג על כל השורות שחולקות את הרגע הזה** – off-by-a-tie. הקלאסי של `keyset`.

התיקון הוא השוואה מורכבת שהיא **טוטלית** מעל `(performed_at, id)`, כי `id` ייחודי:

```
performed_at < ts OR (performed_at = ts AND id < id)
```

והאינדקס ב-0010 נושא את שובר-השוויון באותו סדר שהשאלתה מבקשת, כדי שהחיפוש יישאר range scan ולא יהפוך לסינון.

נמדד על 174 אימונים ב-11 חברים: תשעה עמודים, $14 + 8 \times 20 = 174$ שורות, **אפס חפיפות ואפס דילוגים**, ועמוד 9 רץ ב-3.0ms עם `Index Scan using workouts_havura_feed_idx`.

5. אינדקסים

32 אינדקסים. אלה שנבחרו במכוון:

אינדקס	למה
<code>workouts (havura_id, performed_at desc, id desc)</code>	ה-feed. כולל את שובר-השוויון, כדי שה-cursor המורכב יישאר range scan
<code>workouts (user_id, performed_at desc)</code>	היסטוריה אישית ובייסליין הציון
<code>havura_members (user_id)</code>	ה-PK הוא <code>(havura_id, user_id)</code> ולכן לא משרת את החיפוש ההפוך. זהו הנתבי החם ביותר במערכת – כל policy עובר דרכו
<code>workout_sets (workout_id)</code>	שליפת סטים
<code>arrivals (havura_id, expires_at desc) WHERE closed_at is null</code>	חלקי. מי בחוץ עכשיו – סורק רק הגעות פתוחות
<code>arrivals (user_id, havura_id) WHERE closed_at is null</code>	חלקי. אוקף הגעה פתוחה אחת לחבר לחבורה
<code>create_ledger (user_id, created_at desc)</code>	ה-ledger
<code>create_ledger (user_id, reason, ref_type, ref_id) WHERE ref_id is not null</code>	חלקי. אינו רק ביצועים – הוא מנגנון ה-idempotency
<code>workouts (user_id, source, external_id) WHERE external_id is not null</code>	חוסם ייבוא כפול
<code>inventory (user_id, item_kind) WHERE equipped</code>	אוקף "פריט מצויד אחד מכל סוג"

אינדקס שחסר, ביודעין: `exercises.name` לחיפוש. `ilike '%bench%'` מייצר Seq Scan – נמדד 3.4ms על 660 שורות, ולא ישתנה, כי הקטלוג הוא **קבוע**. 660 שורות סטטיות אינן בעיית סקייל. אם הקטלוג יגדל לעשרות אלפים או ייפתח לתרגילים מותאמים, הפתרון הוא `pg_trgm` עם אינדקס GIN. תוספת אינדקס שלא נמדדה כנחוצה היא עלות כתיבה בלי תמורה.

6. מניעת טעינה מיותרת

מה	איך
קטלוג התרגילים	לא נשלח ללקוח. חיפוש בשרת, עד 25 תוצאות. 660 שורות עם הוראות ביצוע הן בערך 200KB – זו הייתה ברירת המחדל הפשוטה והשגויה
N+1 ב-feed	View אחד עם JOIN במקום שאילתה לכל אימון. 21 round-trips לפרנקפורט הופכים לאחד
שאילתות בלתי תלויות	<code>Promise.all</code> בכל מקום שיש שתיים או יותר. דף הפרופיל מריץ ארבע במקביל
בייסליין הציון	8 שורות בלבד, <code>volume</code> ו- <code>duration_min</code> בלבד
ledger	15 שורות אחרונות, לא הכול
JavaScript	רוב הדפים הם Server Components. ה-bundle כולל רק את הלוגר, מחליף החבורות, כפתורי החנות וטפסים

7. הפרדת לקוח / שרת

Server Components	להגן עליו API ביניים, אין JSON אין Postgres-קוראים ישירות מ
Server Actions	כותבים. ולידציה ולוגיקה עסקית לא ניתנות לעקיפה
Client Components	רק היכן שיש אינטראקטיביות אמיתית

היתרון הסקיילי אינו רק גודל bundle: **אין endpoint לקריאה בכלל**. אין `/api/feed` שצריך rate limiting, אימות ובדיקת הרשאות נפרדת. שטח התקיפה ושטח הביצועים קטנים באותה מידה.

8. עשרות עד מאות משתמשים

בטווח הזה – הטווח שהבריף שואל עליו – **המערכת לא תתקשה**.

משאב	מצב
Postgres	44 שורות בטבלה הגדולה. Supabase Free מספק 500MB; במדידה הנוכחית 500 אימונים למשתמש ל-200 משתמשים עדיין רחוקים מהגבול
חיבורים	Vercel functions דרך ה-pooler. אין חיבור לכל בקשה
Functions	fra1, אותו אזור כמו המסד
Bandwidth	דפים ב-RSC, בלי JSON כפול

החישוב: 200 משתמשים $\times$ 3 אימונים בשבוע = 600 כתיבות בשבוע $\approx$ 3.6 כתיבות בשעה בשיא. קריאות: ~10 טעינות feed למשתמש ליום $\approx$ 2,000 ביום. במדידה של 10ms לשאילתה זה **20 שניות של עבודת מסד ביום**. הצוואר אינו המסד.

9. השאילתות הכבדות באמת

#	שאילתה	סיכון	מתי
1	<code>feed</code> – אגרגציה לפני LIMIT	ליניארי בהיסטוריית החבורה	אלפי אימונים לחבורה
2	<code>weekly_user_stats</code> – אין WHERE על תאריך בתוך ה-view	הוא מאגר את כל ההיסטוריה ואז מסנן לשבוע	שנים של נתונים
3	<code>settleDueWeeks</code> – לולאה עם RPC לכל זוכה	0(חברים) קריאות רשת, בתוך רינדור דף	חבורה גדולה + שבועות רבים שלא נסגרו
4	28 – <code>getMuscleTouches</code> יום של אימונים	חסום בזמן; לא יגדל	–

שאילתה 2 היא אותו דפוס כמו 1 והתיקון דומה: אינדקס על ביטוי התאריך, או טבלה מאוגרת שנכתבת בסגירת השבוע.

9.1 העלות של ה-polling

באנר ההגעות מרענן את עצמו **פעם בדקה** לכל לשונית פתוחה. בחבורה של עשרה שכולם פתוחים בו-זמנית זה 600 בקשות בשעה – נשמע הרבה, ועדיין זניח: כל אחת היא סריקה של אינדקס **חלקי** שמכיל רק הגעות פתוחות, כלומר יחידות ספורות של שורות, ולא הטבלה כולה.

זו הייתה ההצדקה לבחור polling על פני websocket: העלות מדודה וקטנה, והחלופה מוסיפה חיבור מתמיד, מצב כשל נוסף ונושא נוסף במסמך האבטחה. **אם החבורות יגדלו למאות, זו ההחלטה הראשונה שהייתי הופך** – Supabase Realtime על טבלת `arrivals` היא שינוי מקומי, כי האות עצמו כבר מחושב מרשימה.

10. סגירת השבוע – המגבלה הארכיטקטונית הגדולה ביותר

סגירת שבוע מתרחשת כשמישהו פותח את `/crew`. אין cron.

10.1 למה כך

Vercel Hobby לא כולל cron מתוזמן; הוספת שירות תזמון חיצוני הייתה מוסיפה תלות תפעולית שלמה עבור פעולה שרצה פעם בשבוע לחבורה.

10.2 מה זה עולה

השלכה	חומרה
חבורה שאיש לא פתח שלושה שבועות – שלושתם נסגרים ברגע שמישהו כן	תזמון, לא נכונות
הסוגר משלם את המחיר בזמן טעינה	3 שבועות ≈ 3 סבבים של RPC
שתי טעינות במקביל עלולות לנסות לסגור יחד	מוגן: <code>.eq('status', 'open')</code> + אינדקס ה-idempotency ב-ledger. תשלום כפול בלתי אפשרי

הנכונות אינה תלויה בתזמון. רק העדכניות. זו הייתה ההחלטה – ובכוונה תחילה בוצעה כך שהמעבר ל-cron יהיה שינוי של קריאה אחת, לא של הלוגיקה.

10.3 מה הייתי עושה בגרסה הבאה

pg_cron Supabase שקורא לאותה פונקציה בכל יום ראשון ב-00:30 שעות ישראל. הלוגיקה כבר idempotent, אז הפעלה כפולה – ידנית ומתוזמנת – אינה מזיקה.

11. מגבלות ידועות

#	מגבלה	השפעה	תיקון
1	אגרגציה ב-feed לפני LIMIT	ליניארי בהיסטוריה	דנורמליזציה של <code>set_count / volume</code> בכתיבה
2	<code>weekly_user_stats</code> מאגרג הכול	גדל עם הזמן	אינדקס על ביטוי או טבלת אגרגציה
3	cron אין	סגירה מתעכבת	<code>pg_cron</code>
4	סגירה בתוך רינדור דף	הסוגר ממתין	תור, או cron
5	30 proxy ק"ג למשקל גוף	ציון לא מדויק ל-calisthenics	לאסוף משקל גוף
6	אין אינדקס טקסט על <code>exercises.name</code>	Seq Scan	<code>pg_trgm</code> אם הקטלוג יגדל
7	בעלים אינו יכול לעזוב חבורה	הוא תקוע בה עד שתהיה העברת בעלות	חסום ב-UI עם הסבר; העברת בעלות היא הפתרון המלא
8	מתאם Hevy לא מומש	ידני בלבד	ה-API הרשמי; הגבול והאינדקס כבר קיימים
9	אימות אימייל מושבת; המאייל המובנה מוגבל ל-2/שעה	הרשמה מיידית, ללא אימות כתובת	SMTP חיצוני והפעלת האימות מחדש
10	אין rate limiting משלנו	הגנה תלויה ב-Supabase	Postgres ב-Vercel firewall / bucket

12. מה היה משתנה באלף חבורות

#	שינוי	למה
1	דנורמליזציה של סיכומי ה-feed	הבעיה מסעיף 3 הופכת דומיננטית
2	pg_cron לסגירה	הסגירה העצלה מפסיקה להיות סבירה
3	פרטישן של workouts לפי חודש	הטבלה הופכת לגדולה ביותר
4	Cache ל-feed	tenant מבודד → cache פשוט לביטול
5	Read replica	הקריאות עולות על הכתיבות בסדר גודל
6	Rate limiting מפורש	660 שורות בקטלוג × חיפוש בכל הקלדה
7	ניטור אמיתי	היום יש /api/health בלבד

מה שלא היה משתנה: מודל ההרשאות. RLS לפי tenant מתרחב טוב, כי הוא כבר מכריח כל שאילתה להיות מסוננת. זה בדיוק הדבר שהיה קשה להוסיף בדיעבד – ולכן הוא נבנה ראשון.